

Competitive Programming Cheatsheet

David Irvinne

Universitas Gadjah Mada

Daftar Isi

26 struktur data, algoritma & teknik

STRUKTUR DATA	
01 DSU (Union-Find)	3
02 Segment Tree — Range Sum + Lazy	8
03 Segment Tree — Range Max + Lazy	9
04 SRD — Count in Range	3
05 SRD — Count Less-Than	4
06 2D Prefix Sum	4
07 Doubly Linked List	5
GRAPH	
08 LCA (Binary Lifting)	5
09 Ford-Fulkerson (Edmonds-Karp)	6
10 Prim's MST	6
STRING & PARSING	
11 Rabin-Karp	7
12 Expression Evaluator	10
NUMBER THEORY & MATH	
13 Miller-Rabin	7
14 Matrix Operations	11
DIGIT DP	
15 Panjang & Sum Digit Barisan	11
16 Digit DP — Sum di Rentang	12
17 Digit DP — Weighted Sum (pair)	12
18 Digit DP — Divisible by K	13
19 Count Palindromic Numbers	13
DYNAMIC PROGRAMMING	
20 Tower of Hanoi (Rekursif)	14
21 Counting Towers	14
22 0/1 Knapsack — Iteratif	15
23 0/1 Knapsack — Rekursif	15
TEKNIK & KOMBINATORIK	
24 Sliding Window OR (Two-Stack)	16
25 K-th Binary String	16
26 Angular Half-Plane Sort	17

01 · DSU (Union-Find)

STRUKTUR DATA

$O(\alpha(n))$ amortized · scripts/DSU.cpp

```
DSU d(n); d.join(a,b); d.same(a,b);
```

```
struct DSU{
    int n;
    vector<int> par, sz;
    int mx_size = 1, comp;

    DSU(int _n){
        n = _n;
        par.resize(n + 1);
        sz.resize(n + 1);
        comp = _n;
        for(int i = 0; i <= n; i++){
            par[i] = i;
            sz[i] = 1;
        }
    }
    int findRep(int a){
        if(a == par[a]) return a;
        return par[a] = findRep(par[a]);
    }
    bool same(int a, int b){
        return findRep(a) == findRep(b);
    }
    void join(int a, int b){
        a = findRep(a);
        b = findRep(b);
        if (a == b) return;
        if(a > b) swap(a, b); // optional tie-break
        if (sz[a] < sz[b]) swap(a, b);
        par[b] = a;
        sz[a] += sz[b];
        comp--;
    }
};
```

04 · SRD — Count in Range

STRUKTUR DATA

$O(\sqrt{n})$ per query/update · scripts/SRD.cpp · CSES 1144

```
srd.build(); srd.qry(l,r); srd.upd(idx,val);
```

```
// count elements in [l, r], blocks kept sorted
struct square_root_decomposition{
    vector<int> ar;
    vector<vector<int>>> s;
    int sq, n;

    square_root_decomposition(vector<int>&A){
        n = A.size();
        sq = sqrt(n) + 1;
        ar = A;
        s.resize(sq);

    }
    void build(){
        for(int i = 0; i < n; i++) s[i / sq].push_back(ar[i]);
        for(auto&v : s) if(v.size()) sort(v.begin(), v.end());
    }

    int qry(int l, int r){ // count of values within [l, r]
        int res = 0;
        for(auto&v : s){
            int sz = v.size();
            int lb = sz;
            for(int J = 1 << 9; J; J /= 2)
                if(lb - J >= 0 && v[lb - J] >= l) lb -= J;
            int ub = -1;
            for(int J = 1 << 9; J; J /= 2)
                if(ub + J < sz && v[ub + J] <= r) ub += J;
            res += max(0, ub - lb + 1);
        }
        return res;
    }

    void upd(int idx, int new_val){ // idx: 0-based
        int old_val = ar[idx];
        ar[idx] = new_val;
        int block = idx / sq;
        int sz = s[block].size();
        int pos = -1;
        for(int J = 1 << 9; J; J /= 2)
            if(pos + J < sz && s[block][pos + J] <= old_val) pos += J;
        s[block][pos] = new_val;
        int j = pos;
        while(j > 0 && s[block][j-1] > s[block][j]){
            swap(s[block][j-1], s[block][j]);
            j--;
        }
        j = pos;
        while(j + 1 < sz && s[block][j] > s[block][j+1]){
            swap(s[block][j], s[block][j+1]);
            j++;
        }
    }
};
```

05 · SRD — Count Less-Than

STRUKTUR DATA

$O(\sqrt{n} \log n)$ · UVa_12003_Array Transformer.cpp

Beda dari #04: hitung elemen <v pada rentang index, plus update titik.

```
vector<int> A;
vector<vector<int>> block;
int sq;

int cari(vector<int>&vec, int x){ // banyak elemen < x di vec (vec sorted)
    int res = -1, sz = vec.size();
    for(int J = 1 << 15; J; J /= 2)
        if(res + J < sz && vec[res + J] < x) res += J;
    return res + 1;
}

int qry(int l, int r, int v){ // banyak elemen < v pada A[l..r]
    int k = 0;
    int lb = l / sq, rb = r / sq;
    if(lb == rb){
        for(int i = l; i <= r; i++) k += A[i] < v;
        return k;
    }
    for(int i = l; i < (lb + 1) * sq; i++) k += A[i] < v;
    for(int i = lb + 1; i <= rb - 1; i++) k += cari(block[i], v);
    for(int i = rb * sq; i <= r; i++) k += A[i] < v;
    return k;
}

void upd(int pos, int x){ // set A[pos] = x, 0(sqrt(n))
    int old = A[pos];
    A[pos] = x;
    int ppos = pos / sq;
    int idx = -1, sz = block[ppos].size();
    for(int J = 1 << 15; J; J /= 2)
        if(idx + J < sz && block[ppos][idx+J] <= old) idx += J;
    block[ppos][idx] = x;
    while(idx + 1 < sz && block[ppos][idx] > block[ppos][idx+1]){
        swap(block[ppos][idx], block[ppos][idx+1]); idx++;
    }
    while(idx - 1 >= 0 && block[ppos][idx] < block[ppos][idx-1]){
        swap(block[ppos][idx], block[ppos][idx-1]); idx--;
    }
}
```

06 · 2D Prefix Sum

STRUKTUR DATA

$O(nm)$ build, $O(1)$ query · scripts/prefixSum_2d.cpp

```
prefsum_2d pf(ar,n,m); pf.get_sum(r1,c1,r2,c2);
```

```
struct prefsum_2d{
    int n, m;
    vector<vector<int>> pf; // 1-indexed, size (n+1) x (m+1)

    prefsum_2d(vector<vector<int>>&ar, int _n, int _m){
        n = _n;
        m = _m;
        pf = ar;
        build();
    }
    void build(){
        for(int i = 1; i <= n; i++)
            for(int j = 1; j <= m; j++)
                pf[i][j] += pf[i-1][j] + pf[i][j-1] - pf[i-1][j-1];
    }
    int get_sum(int r1, int c1, int r2, int c2){
        if(r1 <= 0) r1 = 1;
        if(r2 <= 0) r2 = 1;
        if(r1 > n) r1 = n;
        if(r2 > n) r2 = n;
        if(c1 <= 0) c1 = 1;
        if(c2 <= 0) c2 = 1;
        if(c1 > m) c1 = m;
        if(c2 > m) c2 = m;
        return pf[r2][c2] - pf[r1-1][c2] - pf[r2][c1-1] + pf[r1-1][c1-1];
    }
};
```

07 · Doubly Linked List

STRUKTUR DATA

O(1) push, O(log n) lookup · scripts/linked_list.cpp

```
struct node{
    node *prv, *nxt;
    int val;
    node(int x){
        val = x;
        prv = nxt = NULL;
    }
};

struct LL{
    node *head, *tail;
    map<int, node*> pos; // value -> node* (assumes unique values)

    LL(){
        head = tail = NULL;
    }

    void push_back(int x){
        node *cur = new node(x);
        if(head == NULL){
            head = tail = cur;
            pos[x] = head;
        } else {
            tail->nxt = cur;
            cur->prv = tail;
            tail = cur;
            pos[x] = tail;
        }
    }

    void push_front(int x){
        node *cur = new node(x);
        if(head == NULL){
            head = tail = cur;
            pos[x] = head;
        } else {
            head->prv = cur;
            cur->nxt = head;
            head = cur;
            pos[x] = head;
        }
    }

    void print(){
        node *now = head;
        while(now != NULL){
            cout << (now->val) << ' ';
            now = now->nxt;
        }
        cout << '\n';
    }
};
```

08 · LCA (Binary Lifting)

GRAPH

O(n log n) build, O(log n) query · scripts/LCA.cpp

```
struct LCA {
    vector<vector<int>> anc;
    vector<int> depth;
    int N, LOG;

    LCA(int n, int root, vector<vector<int>>&adj){
        N = n;
        LOG = 1;
        while((1ll << LOG) <= N) LOG++;
        anc.resize(N + 1, vector<int>(LOG));
        depth.resize(N + 1);
        fill_anc(root, 0, adj);
    }

    int get_lca(int a, int b){
        if(depth[a] > depth[b]) swap(a, b); // buat b lebih dalam
        int k = depth[b] - depth[a];
        for(int j = LOG - 1; j >= 0; j--){
            if(k >= (1 << j)){
                k -= (1 << j);
                b = anc[b][j];
            }
        }
        if(a == b) return a;
        for(int j = LOG - 1; j >= 0; j--){
            if(anc[a][j] != anc[b][j]){
                a = anc[a][j];
                b = anc[b][j];
            }
        }
        return anc[a][0];
    }

    void fill_anc(int now, int par, vector<vector<int>>&adj){
        anc[now][0] = par;
        for(int i = 1; i < LOG; i++)
            anc[now][i] = anc[anc[now][i-1]][i-1];
        depth[now] = depth[par] + 1;
        for(int nxt : adj[now]){
            if(nxt == par) continue;
            fill_anc(nxt, now, adj);
        }
    }
};
```

09 · Ford–Fulkerson

GRAPH · MAX FLOW

$O(V \cdot E^2)$ · scripts/ford_fulkerson.cpp

Residual back-edge (rg[v][u]+=path_flow) di-nonaktifkan di source asli — aktifkan utk graf umum.

```
int n, m; // n nodes, m edges
int g[MAXN][MAXN], rg[MAXN][MAXN], parent[MAXN];

// true jika ada jalan dari s ke t di residual graph
bool bfs(int s, int t){
    vector<bool> vis(n);
    queue<int> q;
    q.push(s);
    vis[s] = 1;
    parent[s] = -1;

    while(q.size()){
        int u = q.front();
        q.pop();
        for(int v = 0; v < n; v++){
            if(!vis[v] && rg[u][v] > 0){
                q.push(v);
                parent[v] = u;
                vis[v] = 1;
            }
        }
    }
    return vis[t];
}

int max_flow(int s, int t){
    int max_flow = 0;
    while(bfs(s, t)){
        int path_flow = INT_MAX;
        for(int v = t; v != s; v = parent[v]){
            int u = parent[v];
            path_flow = min(path_flow, rg[u][v]);
        }
        for(int v = t; v != s; v = parent[v]){
            int u = parent[v];
            rg[u][v] -= path_flow;
            // rg[v][u] += path_flow; // aktifkan utk residual back-edge yg benar
        }
        max_flow += path_flow;
    }
    return max_flow;
}
```

10 · Prim's MST

GRAPH

$O(E \log V)$ · scripts/prim.cpp

```
int n, m;
vector<vector<pair<int,int>>> adj(n + 1); // adj[u] = {v, w}
// baca edge: adj[u].push_back({v,w}); adj[v].push_back({u,w});

int prim_mst(int start = 1){
    priority_queue<pair<int,int>> pq; // {-cost, node} agar jadi min-heap
    for(auto&[v, w] : adj[start]) pq.push({-w, v});

    vector<bool> vis(n + 1);
    vis[start] = true;
    int mst = 0, cnt = 1;

    while(pq.size()){
        auto [w, v] = pq.top();
        pq.pop();
        if(vis[v]) continue;

        vis[v] = true;
        cnt++;
        mst += -w;
        for(auto&[dest, cost] : adj[v]) pq.push({-cost, dest});
    }
    return (cnt == n) ? mst : -1; // -1 jika graf tidak terhubung
}
```

11 · Rabin–Karp

STRING

$O(n)$ build, $O(1)$ hash · scripts/rabin_karp.cpp

Perlu bpow/inv (kartu sebelah).

```
struct RabinKarp{
    static const int MOD = 1e9 + 9;
    int PP = 53;
    vector<int> H, P, INVP; // 1-based index
    string S;

    RabinKarp(){}

    RabinKarp(string s, int _PP = 53){
        S = " " + s;
        PP = _PP;
        int sz = s.size();
        H = vector<int>(sz + 1);
        P = vector<int>(sz + 1);
        INVP = vector<int>(sz + 1);

        P[0] = 1;
        for(int i = 1; i <= sz; i++){
            P[i] = (P[i-1] * PP) % MOD;
            INVP[i] = inv(P[i], MOD);
        }
        for(int i = 1; i <= sz; i++){
            H[i] = (H[i-1] + (S[i] - 'a' + 1) * P[i]) % MOD;
        }
    }

    int get_hash(int l, int r){ // 1-based, inclusive
        int res = (H[r] - H[l-1] + MOD) % MOD;
        res *= INVP[l];
        res %= MOD;
        return res;
    }
};
```

13 · Miller–Rabin + bpow/inv

NUMBER THEORY

$O(k \log^3 n)$ · scripts/miller_rabin.cpp · deterministik $<3.3 \times 10^{-2}$ &sup4; via witness {2..37}

MillerRabin(n); bpow(a,b,mod); inv(a,mod);

```
using u64 = uint64_t;
using u128 = __uint128_t;

u64 binpower(u64 base, u64 e, u64 mod) {
    u64 result = 1;
    base %= mod;
    while (e) {
        if (e & 1) result = (u128)result * base % mod;
        base = (u128)base * base % mod;
        e >>= 1;
    }
    return result;
}

bool check_composite(u64 n, u64 a, u64 d, int s) {
    u64 x = binpower(a, d, n);
    if (x == 1 || x == n - 1) return false;
    for (int r = 1; r < s; r++) {
        x = (u128)x * x % n;
        if (x == n - 1) return false;
    }
    return true;
}

// true jika n kemungkinan besar prima
bool MillerRabin(u64 n, int iter = 5) {
    if (n < 4) return n == 2 || n == 3;
    int s = 0;
    u64 d = n - 1;
    while ((d & 1) == 0) { d >>= 1; s++; }

    for (int i = 0; i < iter; i++) {
        int a = 2 + rand() % (n - 3);
        if (check_composite(n, a, d, s)) return false;
    }
    return true;
}

long long bpow(long long a, long long b, long long mod = LLONG_MAX){
    long long res = 1;
    a %= mod;
    while(b){
        if(b & 1) res = res * a % mod;
        a = a * a % mod;
        b >>= 1;
    }
    return res;
}

long long inv(long long a, long long mod = 1e9 + 7){
    return bpow(a, mod - 2, mod); // syarat: mod bilangan prima (Fermat)
}
```

02 · Segment Tree — Range Sum + Lazy Propagation

$O(\log n)$ per query/update · scripts/lazy_propagation.cpp · lazy value dibagi 2 ke tiap anak (distribusi merata, sesuaikan utk semantik lain)

```
segment_tree<long long> st(arr, true); st.range_sum(l,r); st.range_update(l,r,val); st.point_update(i,val);
```

```
// padded segment tree, merged with lazy propagation
template<typename T>
struct segment_tree {
    vector<T> tree, lz;
    int sz = 1;
    bool lazy_propagation = false;

    segment_tree(vector<T>&ar, bool lp = false){
        lazy_propagation = lp;
        int n = ar.size();
        while(sz < n) sz *= 2;
        tree.resize(2 * sz);
        if(lazy_propagation) lz.resize(2 * sz);
        build(ar);
    }

    void build(vector<T>&ar){
        int n = ar.size();
        for(int i = 0; i < n; i++) tree[sz + i] = ar[i];
        for(int i = sz - 1; i >= 1; i--){
            tree[i] = merge(tree[2 * i], tree[2 * i + 1]);
        }

        T merge(T&a, T&b){ return a + b; }

        T range_sum(int ql, int qr, int t, int tl, int tr){
            if(lazy_propagation && lz[t]){
                tree[t] += lz[t];
                if(tl != tr){
                    lz[2 * t] += lz[t] / 2;
                    lz[2 * t + 1] += lz[t] / 2;
                }
                lz[t] = 0;
            }
            if(ql <= tl && tr <= qr) return tree[t];
            if(tl > qr || ql > tr) return 0;
            int mid = (tl + tr) / 2;
            return range_sum(ql, qr, 2 * t, tl, mid)
                + range_sum(ql, qr, 2 * t + 1, mid + 1, tr);
        }

        T range_sum(int ql, int qr){
```

```
            return range_sum(ql, qr, 1, 0, sz - 1);
        }

        void point_update(int idx, T new_val){
            idx += sz;
            tree[idx] = new_val;
            for(idx /= 2; idx >= 1; idx /= 2)
                tree[idx] = merge(tree[2 * idx], tree[2 * idx + 1]);
        }

        void range_update(int ql, int qr, int val, int t, int tl, int tr){
            if(lz[t]){
                tree[t] += lz[t];
                if(tl != tr){
                    lz[2 * t] += lz[t] / 2;
                    lz[2 * t + 1] += lz[t] / 2;
                }
                lz[t] = 0;
            }
            if(ql <= tl && tr <= qr){
                tree[t] += val * (tr - tl + 1);
                if(tl != tr){
                    lz[2 * t] += val * (tr - tl + 1) / 2;
                    lz[2 * t + 1] += val * (tr - tl + 1) / 2;
                }
                return;
            }
            if(qr < tl || ql > tr) return;
            int mid = (tl + tr) / 2;
            range_update(ql, qr, val, 2 * t, tl, mid);
            range_update(ql, qr, val, 2 * t + 1, mid + 1, tr);
            tree[t] = tree[2 * t] + tree[2 * t + 1];
        }

        void range_update(int ql, int qr, int val){
            if(!lazy_propagation){
                cout << "ERROR: MUST IMPLEMENT LAZY PROPAGATION\n";
                return;
            }
            range_update(ql, qr, val, 1, 0, sz - 1);
        }
    };
};
```


03 · Segment Tree — Range Max + Lazy Propagation

STRUKTUR DATA

$O(\log n)$ per query/update · online_judge/prefix_sum_queries.cpp · lazy TIDAK dibagi 2 (beda dari hal. 8) — dipakai maintain prefix-max via point-update

```
segment_tree<int> st(ar, true); st.range_query(l,r); st.range_update(l,r,val);
```

```
template<typename T>
struct segment_tree {
    vector<T> tree, lz;
    int sz = 1;
    bool lazy_propagation = false;

    segment_tree(vector<T>&ar, bool lp = false){
        lazy_propagation = lp;
        int n = ar.size();
        while(sz < n) sz *= 2;
        tree.resize(2 * sz);
        if(lazy_propagation) lz.resize(2 * sz);
        build(ar);
    }

    void build(vector<T>&ar){
        int n = ar.size();
        for(int i = 0; i < n; i++) tree[sz + i] = ar[i];
        for(int i = sz - 1; i >= 1; i--){
            tree[i] = merge(tree[2 * i], tree[2 * i + 1]);
        }
    }

    T merge(T a, T b){ return max(a, b); }

    T range_query(int ql, int qr, int t, int tl, int tr){
        if(lazy_propagation && lz[t]){
            tree[t] += lz[t];
            if(tl != tr){
                lz[2 * t] += lz[t];
                lz[2 * t + 1] += lz[t];
            }
            lz[t] = 0;
        }
        if(ql <= tl && tr <= qr) return tree[t];
        if(tl > qr || ql > tr) return -(T)2e15; // default outside value

        int mid = (tl + tr) / 2;
        return merge(range_query(ql, qr, 2*t, tl, mid), range_query(ql, qr,
    T range_query(int ql, int qr){ return range_query(ql, qr, 1, 0, sz -
    1); }

    void point_update(int idx, T new_val){
        idx += sz;
        tree[idx] = new_val;
        for(idx /= 2; idx >= 1; idx /= 2)
            tree[idx] = merge(tree[2*idx], tree[2*idx+1]);
    }

    void range_update(int ql, int qr, int val, int t, int tl, int tr){
        if(lz[t]){
            tree[t] += lz[t];
            if(tl != tr){ lz[2*t] += lz[t]; lz[2*t+1] += lz[t]; }
            lz[t] = 0;
        }
        if(ql <= tl && tr <= qr){
            tree[t] += val;
            if(tl != tr){ lz[2*t] += val; lz[2*t+1] += val; }
            return;
        }
        if(qr < tl || ql > tr) return;
        int mid = (tl + tr) / 2;
        range_update(ql, qr, val, 2*t, tl, mid);
        range_update(ql, qr, val, 2*t+1, mid+1, tr);
        tree[t] = merge(tree[2*t], tree[2*t+1]);
    }

    void range_update(int ql, int qr, int val){
        range_update(ql, qr, val, 1, 0, sz - 1);
    }
};
```

12 · Expression Evaluator (Tokenizer + Shunting-Yard + RPN)

STRING & PARSING

O(n) tokenize + parse + eval · online_judge/eval_math.cpp · mendukung + - * / ** (kanan-assoc), kurung, unary minus

```
auto rpn = toRPN(tokenize(expr)); long long hasil = evalRPN(rpn);
```

```
long long ipow(long long base, long long exp){
    long long res = 1, b = base;
    while(exp){
        if(exp & 1) res = res * b;
        b = b * b;
        exp >>= 1;
    }
    return res;
}

bool isOperator(const string &t){
    return t=="+" || t=="-" || t=="*" || t=="/" || t=="**";
}

int prec(const string &op){
    if(op=="**") return 4;
    if(op=="*" || op=="/") return 3;
    if(op=="+" || op=="-") return 2;
    return 0;
}

bool isRightAssoc(const string &op){ return op == "**"; }

// tokenisasi angka, +-*/**, tanda kurung, dan unary minus
vector<string> tokenize(const string &s){
    vector<string> tokens;
    int n = s.size(), i = 0;
    bool prevIsOp = true;
    while(i < n){
        unsigned char ch = s[i];
        if(isspace(ch)){ i++; continue; }
        if(isdigit(ch)){
            int j = i;
            while(j < n && isdigit((unsigned char)s[j])) j++;
            tokens.push_back(s.substr(i, j - i));
            i = j; prevIsOp = false;
        } else if(s[i]=='('){ tokens.push_back("("); i++; prevIsOp = true; }
        else if(s[i]==')'){ tokens.push_back(")"); i++; prevIsOp = false; }
        else if(s[i]=='+' || s[i]=='-'){
            if(prevIsOp || tokens.empty() || tokens.back()=="("){
                if(s[i]=='-'){ i++; continue; }
                else { tokens.push_back("0"); tokens.push_back("-"); i++; prevIsOp = true; }
            } else { tokens.push_back(string(1,s[i])); i++; prevIsOp = true; }
        } else if(s[i]=='*'){
            if(i+1<n && s[i+1]=='*'){ tokens.push_back("**"); i+=2; }
            else { tokens.push_back("*"); i++; }
            prevIsOp = true;
        } else if(s[i]=='/'){ tokens.push_back("/"); i++; prevIsOp = true; }
        else if(s[i]=='^'){ tokens.push_back("**"); i++; prevIsOp = true; } // caret = power
        else throw runtime_error("unexpected character");
    }
    return tokens;
}

// Shunting-Yard: infix -> Reverse Polish Notation
vector<string> toRPN(const vector<string> &tokens){
    vector<string> output, ops;
    for(const string &t : tokens){
        if(isdigit((unsigned char)t[0])) output.push_back(t);
        else if(isOperator(t)){
            while(!ops.empty() && isOperator(ops.back())){
                string top = ops.back();
                if(!isRightAssoc(t) && prec(t) <= prec(top)) ||
                   (isRightAssoc(t) && prec(t) < prec(top)){
                    output.push_back(top); ops.pop_back();
                } else break;
            }
            ops.push_back(t);
        } else if(t == "("){ ops.push_back(t); }
        else if(t == ")"){
            while(!ops.empty() && ops.back() != "("){ output.push_back(ops.back());
                ops.pop_back(); } // buang "("
            ops.pop_back();
        }
    }
    while(!ops.empty()){ output.push_back(ops.back()); ops.pop_back(); }
    return output;
}

long long evalRPN(const vector<string> &rpn){
    vector<long long> st;
    for(const string &t : rpn){
        if(isdigit((unsigned char)t[0])) st.push_back(stoll(t));
        else {
            long long b = st.back(); st.pop_back();
            long long a = st.back(); st.pop_back();
            long long res = 0;
            if(t=="+") res=a+b;
            else if(t=="-") res=a-b;
            else if(t=="**") res=a*b;
            else if(t=="/") res=a/b;
            else if(t=="**") res=ipow(a,b);
            st.push_back(res);
        }
    }
    return st.back();
}
```

14 · Matrix Operations

MATH

$O(k^3 \log n)$ · scripts/matrix_operation.cpp · perlu MOD global

```
vector<vector<int>> mul(vector<vector<int>> left, vector<vector<int>> right){
    vector<vector<int>> res;
    for(int i = 0; i < left.size(); i++){
        res.push_back({});
        for(int j = 0; j < right[0].size(); j++){
            int sum = 0;
            for(int k = 0; k < left[0].size(); k++){
                sum += left[i][k] * right[k][j];
                sum = (sum + MOD) % MOD;
            }
            res.back().push_back(sum);
        }
    }
    return res;
}

vector<vector<int>> mpow(vector<vector<int>> a, int b){
    vector<vector<int>> res = { {1, 0}, {0, 1} }; // identity 2x2
    while(b){
        if(b % 2) res = mul(res, a);
        a = mul(a, a);
        b /= 2;
    }
    return res;
}
```

15 · Panjang & Sum Digit Barisan

DIGIT DP

$O(\log^2 n)$ · online_judge/1043d.cpp · bit-jump search

```
int f(int n){ // panjang digit dari "1234...n" (concatenation)
    int mx_len = to_string(n).size();
    int total = 0;
    for(int len = 1; len < mx_len; len++){
        total += len * 9ll * bpow(10ll, len - 1);
        if(total > (int)1e15) return LLONG_MAX;
    }
    total += mx_len * (n - bpow(10ll, mx_len - 1) + 1);
    return total;
}

int sum_digits_1_to_n(int n){ // sum semua digit dari 1 s/d n, closed form
    if(n <= 0) return 0;
    __int128_t total = 0, p = 1, N = n;
    while(p <= N){
        __int128_t left = N / (p * 10);
        int cur = (int)((N / p) % 10);
        __int128_t right = N % p;
        total += left * 45 * p;
        total += (__int128_t)cur * (cur - 1) / 2 * p;
        total += (__int128_t)cur * (right + 1);
        p *= 10;
    }
    return (int)total;
}

// cari k-digit pertama dari string "1234...n...", jumlahkan digitnya
int solve(int k){
    int h = 0;
    for(int J = 1ll << 60; J; J /= 2) // bit-jump binary search (predikat monoton)
        if(f(h + J) <= k) h += J;

    int sum = sum_digits_1_to_n(h);
    if(f(h) != k){
        int len = k - f(h);
        string tmp = to_string(h + 1);
        for(int i = 0; len--; i++) sum += tmp[i] - '0';
    }
    return sum;
}
```

16 · Digit DP — Sum of Digits di Rentang

DIGIT DP

SPOJ CPCRC1C

```
solve(r) - solve(l-1); // jumlah semua digit bilangan di [l,r]
```

```
vector<int> digits;
int memo[11][2][9*11];

int dp(int idx, bool tight, int sum){
    if(idx == (int)digits.size()) return sum;
    if(memo[idx][tight][sum] != -1) return memo[idx][tight][sum];
    int limit = tight ? digits[idx] : 9;
    int res = 0;
    for(int i = 0; i <= limit; i++){
        res += dp(idx + 1, tight && i == limit, sum + i);
    }
    return memo[idx][tight][sum] = res;
}

int solve(int x){ // jumlah semua digit bilangan 0..x
    memset(memo, -1, sizeof(memo));
    digits.clear();
    if(x == 0) digits.push_back(0);
    while(x > 0){ digits.push_back(x % 10); x /= 10; }
    reverse(digits.begin(), digits.end());
    return dp(0, true, 0);
}

// jawaban [l,r]: solve(r)-solve(l-1)
```

17 · Digit DP — Weighted Sum (pair)

DIGIT DP

SPOJ PR003004

```
solve(R) - solve(L-1); // total penjumlahan seluruh digit di [L,R]
```

```
pair<int,int> dp[17][2]; // {total sum digit, banyak cara}

pair<int,int> f(int idx, bool tight, string &s){
    if(idx == (int)s.size()) return {0, 1};
    if(dp[idx][tight].first != -1) return dp[idx][tight];
    int sum = 0, way = 0, limit = tight ? s[idx] - '0' : 9;
    for(int i = 0; i <= limit; i++){
        auto [subsum, subway] = f(idx + 1, tight && i == limit, s);
        way += subway;
        sum += i * subway + subsum;
    }
    return dp[idx][tight] = {sum, way};
}

int solve(int x){
    string s = to_string(x);
    for(int i = 0; i < 17; i++){
        for(int j = 0; j < 2; j++){ dp[i][j] = {-1, -1}; }
    }
    return f(0, true, s).first;
}

// jawaban [L,R]: solve(R)-solve(L-1)
```

18 · Digit DP — Divisible by K

DIGIT DP

lightoj.com/problem/investigation

```
int dp[10][99][2][99];
vector<int> digits;
int k;

// idx, current number mod k, tight, current digit-sum mod k
int f(int idx, int cur, bool tight, int sum){
    if(idx == (int)digits.size()) return cur % k == 0 && sum % k == 0;
    if(dp[idx][cur][tight][sum] != -1) return dp[idx][cur][tight][sum];
    int res = 0, limit = tight ? digits[idx] : 9;
    for(int i = 0; i <= limit; i++){
        res += f(idx + 1, (cur * 10 + i) % k, tight && i == limit, (sum + i) % k);
    }
    return dp[idx][cur][tight][sum] = res;
}

int solve(int x){ // banyak bilangan 0..x habis dibagi k DAN digit-sum habis dibagi k
    if(k > 90) return 0; // digit-sum maksimal < 90
    memset(dp, -1, sizeof(dp));
    digits.clear();
    if(x == 0) digits.push_back(0);
    while(x > 0){ digits.push_back(x % 10); x /= 10; }
    reverse(digits.begin(), digits.end());
    return f(0, 0, true, 0);
}
// jawaban [l,r]: solve(r)-solve(l-1)
```

19 · Palindromic Numbers ≤ X

DIGIT DP · MATH

O(log x) · lightoj.com/problem/palindromic-numbers

```
int solve(int x){ // banyak bilangan palindrom pada [0, x]
    if(x <= 10) return min(x + 1, 10ll);
    int len = to_string(x).size();
    string s = to_string(x);
    if(x % 10 == 0){ s = to_string(x - 1); len = s.size(); }

    int res = 10; // hitung semua palindrom dengan panjang < len
    for(int l = 2, add = 9; l < len; l++){
        if(l % 2) add *= 10;
        res += add;
    }
    // sesuaikan half pertama <= half kedua (biar palindrom <= x)
    for(int i = len - 1, pp = 0; i > len / 2; i--, pp++){
        if(s[i] < s[len - i - 1]){
            x -= (s[i] - '0' + 1) * bpow(10ll, pp);
            s = to_string(x);
        }
    }
    res += stoll(s.substr(0, (len + 1) / 2));
    res -= bpow(10ll, (len + 1) / 2 - 1);
    return res;
}
// jawaban [L,R]: solve(R)-solve(L-1)
```

20 · Tower of Hanoi

DYNAMIC PROGRAMMING

$O(2^{\sup n})$ · cses_tower_of_hanoi.cpp

```
void solve(int a, int b, int c, int n){ // pindahkan n disk a->c, lewat b
    if(n == 0) return;
    solve(a, c, b, n - 1);
    cout << a << ' ' << c << '\n'; // pindah disk teratas a->c
    solve(b, a, c, n - 1);
}
// total langkah minimum:  $2^n - 1$ 
```

21 · Counting Towers

DYNAMIC PROGRAMMING

$O(n)$ precompute, $O(1)$ /query

```
const int MOD = 1e9 + 7, N = 1e6 + 2;
vector<int> one(N + 2), two(N + 2);
// one[i]: cara isi papan lebar 2*i, sisi kanan "rata"
// two[i]: cara isi papan lebar 2*i, sisi kanan "tidak rata"
one[1] = two[1] = 1;
for(int i = 2; i <= N; i++){
    one[i] = (2 * one[i-1] + two[i-1]) % MOD;
    two[i] = (4 * two[i-1] + one[i-1]) % MOD;
}
// jawaban lebar x: (one[x] + two[x]) % MOD
```

22 · 0/1 Knapsack — Iteratif

DYNAMIC PROGRAMMING

O(n·bag) · [online_judge/tlx_knapsack.cpp](#)

+ rekonstruksi item terpilih (backtrack tabel dp).

```
int w[N], h[N], dp[N][N];

void solve(){
    int bag, n;
    cin >> bag >> n;
    for(int i = 1; i <= n; i++) cin >> w[i] >> h[i];

    for(int i = 1; i <= n; i++){
        for(int j = 0; j <= bag; j++){
            int res = dp[i-1][j];
            if(j >= w[i]) res = max(res, dp[i-1][j-w[i]] + h[i]);
            dp[i][j] = res;
        }
    }

    int min_w = bag;
    for(int i = bag - 1; i >= 0; i--){
        if(dp[n][bag] == dp[n][i]) min_w = i;
    }

    vector<int> c;
    for(int i = n; i > 0 && min_w > 0; i--){
        if(dp[i][min_w] != dp[i-1][min_w]){
            min_w -= w[i];
            c.push_back(i);
        }
    }
    for(int i = c.size() - 1; i >= 0; i--) cout << c[i] << '\n';
}
```

23 · 0/1 Knapsack — Rekursif

DYNAMIC PROGRAMMING

O(n·bag) memoized · [tlx_knapsack.cpp](#)

```
int w[N], h[N], k, n, dp[N][N];

int f(int i, int bag){
    if(i < 0) return 0;
    if(dp[i][bag] != -1) return dp[i][bag];
    int res = f(i - 1, bag);
    if(bag >= w[i]) res = max(res, f(i - 1, bag - w[i]) + h[i]);
    return dp[i][bag] = res;
}

void solve(){
    cin >> n >> k;
    for(int i = 0; i < k; i++) cin >> w[i] >> h[i];
    memset(dp, -1, sizeof(dp));
    f(k - 1, n);

    int x = k - 1, y = n;
    vector<int> res;
    while(x >= 0 && y >= 0){
        if(y >= w[x] && f(x - 1, y - w[x]) + h[x] == dp[x][y]){
            res.push_back(x);
            y -= w[x];
        }
        x--;
    }
    reverse(res.begin(), res.end());
    for(int i : res) cout << i + 1 << ' ';
}
```

24 · Sliding Window OR

TEKNIK

O(1) amortized · cses_sliding_or.cpp

Two-stack: berlaku utk op asosiatif idempoten (OR/AND/GCD/MIN/MAX).

```
// dua stack: "masuk" (elemen baru), "keluar" (elemen lama, siap pop)
// tiap entri {nilai, OR-prefix-dari-stack}
vector<array<int,2>> masuk, keluar;

void push(int val){
    if(masuk.empty()) masuk.push_back({val, val});
    else masuk.push_back({val, masuk.back()[1] | val});
}

void pop(){
    if(keluar.empty()){
        while(masuk.size()){
            auto now = masuk.back(); masuk.pop_back();
            if(keluar.empty()) keluar.push_back({now[0], now[0]});
            else keluar.push_back({now[0], keluar.back()[1] | now[0]});
        }
    }
    keluar.pop_back();
}

int window_or(){ // OR seluruh isi window sekarang
    int r = 0;
    if(masuk.size()) r |= masuk.back()[1];
    if(keluar.size()) r |= keluar.back()[1];
    return r;
}
```

25 · K-th Binary String

KOMBINATORIK

O(len)/query · gemastik24_biner.cpp & gemastik_biner.cpp (identik)

```
int C[60][60]; // C[i][j] = pilih j dari i, Pascal's triangle
for(int i = 1; i < 60; i++)
    for(int j = 0; j <= i; j++)
        C[i][j] = (j == 0 || i == j) ? 1 : C[i-1][j] + C[i-1][j-1];

// contoh: cari representasi biner ke-n (1-indexed, 30 bit, awalan '1')
int cnt0 = 0;
while(C[cnt0 + 29][cnt0] < n){ n -= C[cnt0 + 29][cnt0]; cnt0++; }

vector<int> res;
int cnt1 = 29;
res.push_back(1);
while(cnt0 + cnt1 > 0){
    if(cnt0 == 0){ res.push_back(1); cnt1--; continue; }
    if(cnt1 == 0){ res.push_back(0); cnt0--; continue; }

    if(C[cnt0 + cnt1 - 1][cnt0 - 1] < n){ // '0' di posisi ini tak cukup ruang
        n -= C[cnt0 + cnt1 - 1][cnt0 - 1];
        cnt1--; res.push_back(1);
    } else { cnt0--; res.push_back(0); }
}
```


26 · Angular Half-Plane Sort + Range Counting

GEOMETRI

`O(n log n)` sort · laser_takahashi.cpp · atcoder.jp/contests/abc442/tasks/abc442_e

```
sort(v.begin(), v.end(), angleCmp); // lalu simpan first_pos/last_pos per arah unik
```

```
struct vec {
    int x, y;
    int half(){ return (y > 0 || (y == 0 && x > 0)); } // 1 = upper half-plane
    void simp(){ int g = gcd(abs(x), abs(y)); x /= g; y /= g; }
};

int cross(vec a, vec b){ return a.x * b.y - a.y * b.x; }

// urutkan vector berdasarkan sudut (CCW), half-plane atas duluan
sort(v.begin(), v.end(), [&](vec l, vec r){
    if(l.half() != r.half()) return l.half() > r.half();
    return cross(l, r) > 0;
});

// setelah sort: catat first/last occurrence tiap arah unik (v[i].simp())
// utk menjawab query "berapa banyak vector antara arah A dan B (CCW)"
// secara O(1) per query.
```